

# **GYMBRO**

Sistema de Detecção de Polichinelos com Feedback Físico  
Utilizando Acelerômetro, Câmera e Microcontrolador ESP32

2026

## RESUMO

O presente relatório descreve o desenvolvimento do **GymBro**, um sistema integrado de detecção de polichinelos (*jumping jacks*) que combina sensores inerciais de um dispositivo Android, visão computacional via MediaPipe Pose, e uma estação de feedback físico baseada no microcontrolador ESP32 simulado no ambiente Wokwi. O sistema adota uma arquitetura orientada a eventos na qual o telefone envia os resultados da detecção para um retransmissor (*relay*) HTTP na nuvem (Railway), que por sua vez é consultado periodicamente pelo ESP32. Um *worker* do Cloudflare faz a ponte entre o protocolo HTTP (usado pelo ESP32) e o HTTPS (exigido pela plataforma Railway), permitindo que o microcontrolador, com recursos limitados de *software*, obtenha os dados sem a necessidade de TLS. A estação ESP32 exibe as informações em um display LCD 20×4 I2C, emite sinais sonoros via buzzer a cada repetição, mede a distância do usuário com sensor ultrassônico HC-SR04. O aplicativo Android oferece dois modos de detecção: sensor (acelerômetro + giroscópio) e câmera (MediaPipe Pose Landmarker). Cujos resultados são combinados por um classificador de estados. O sistema foi validado em ambiente simulado, demonstrando a viabilidade da arquitetura proposta para aplicações de *fitness* com *feedback* físico em tempo real.

**Palavras-chave:** Detecção de polichinelos. ESP32. MediaPipe Pose. Acelerômetro. Wokwi. Cloudflare Workers. Retransmissor HTTP. Feedback físico.



## LISTA DE FIGURAS

|                                                            |    |
|------------------------------------------------------------|----|
| Figura 1 — Arquitetura geral do sistema GymBro .....       | 8  |
| Figura 2 — Ciclo de detecção do aplicativo Android .....   | 10 |
| Figura 3 — Diagrama de blocos do <i>relay</i> HTTP .....   | 11 |
| Figura 4 — Fluxo do <i>worker</i> Cloudflare .....         | 12 |
| Figura 5 — Diagrama de componentes do ESP32 no Wokwi ..... | 13 |
| Figura 6 — Ciclo principal do <i>firmware</i> ESP32 .....  | 14 |
| Figura 7 — Fluxo completo de dados do sistema .....        | 15 |

## LISTA DE TABELAS

|                                                          |    |
|----------------------------------------------------------|----|
| Tabela 1 — Componentes de <i>hardware</i> do ESP32 ..... | 13 |
| Tabela 2 — Endpoints do <i>relay</i> HTTP .....          | 11 |
| Tabela 3 — Endpoints do servidor embutido Android .....  | 10 |



## 1 INTRODUÇÃO

A prática regular de exercícios físicos é fundamental para a manutenção da saúde e do bem-estar. Entre os exercícios aeróbicos mais comuns, o polichinelo (*jumping jack*) destaca-se por sua simplicidade e eficácia. No entanto, a automonitoração da quantidade e da qualidade das repetições pode ser imprecisa quando feita manualmente, o que motiva o desenvolvimento de sistemas automatizados de detecção e contagem.

Este relatório apresenta o **GymBro**, um sistema que integra um aplicativo Android, um retransmissor na nuvem e uma estação de *feedback* físico baseada no microcontrolador ESP32 para detectar, contabilizar e exibir polichinelos em tempo real. O sistema utiliza duas modalidades de detecção no telefone — sensores inerciais (acelerômetro e giroscópio) e visão computacional (MediaPipe Pose Landmarker), e encaminha os resultados para um *relay* HTTP na plataforma Railway. O ESP32, executado no simulador Wokwi, consulta periodicamente esse *relay* por intermédio de um *worker* Cloudflare que converte a requisição HTTP em HTTPS, e exibe as informações em um display LCD 20×4, emitindo sinais sonoros a cada repetição.

O documento está organizado em nove seções. A Seção 2 descreve a arquitetura geral do sistema. As Seções 3, 4, 5 e 6 detalham, respectivamente, o aplicativo Android, o *relay* HTTP, o *worker* Cloudflare e o *firmware* do ESP32. A Seção 7 apresenta a integração e o fluxo de dados completo. A Seção 8 discute os resultados obtidos, e a Seção 9 traz as conclusões e direções para trabalhos futuros.





## 2 ARQUITETURA DO SISTEMA

O GymBro adota uma arquitetura de três camadas com comunicação orientada a eventos e sondagem periódica (*polling*). A camada de aquisição e classificação reside no aplicativo Android, que coleta dados dos sensores inerciais e da câmera, processa-os por meio de um classificador de estados e produz um objeto `JackResult` contendo os campos `isJacking`, `repCount`, `formQuality` e `confidence`.

A camada de retransmissão é composta por dois serviços: (i) um *relay* HTTP escrito em Python com a biblioteca padrão (`http.server`), implantado na plataforma Railway, que aceita requisições POST do telefone e requisições GET do ESP32; e (ii) um *worker* Cloudflare que atua como *proxy* de protocolo, recebendo requisições HTTP do ESP32 e as redirecionando ao *relay* via HTTPS.

A camada de *feedback* físico é o microcontrolador ESP32 simulado no Wokwi, que executa um *firmware* em MicroPython. O ESP32 sonda o *worker* Cloudflare a cada segundo, processa a resposta JSON e atualiza o display LCD 20×4, o buzzer e o contador de distância ultrassônico. A Figura 1 ilustra a arquitetura geral do sistema.

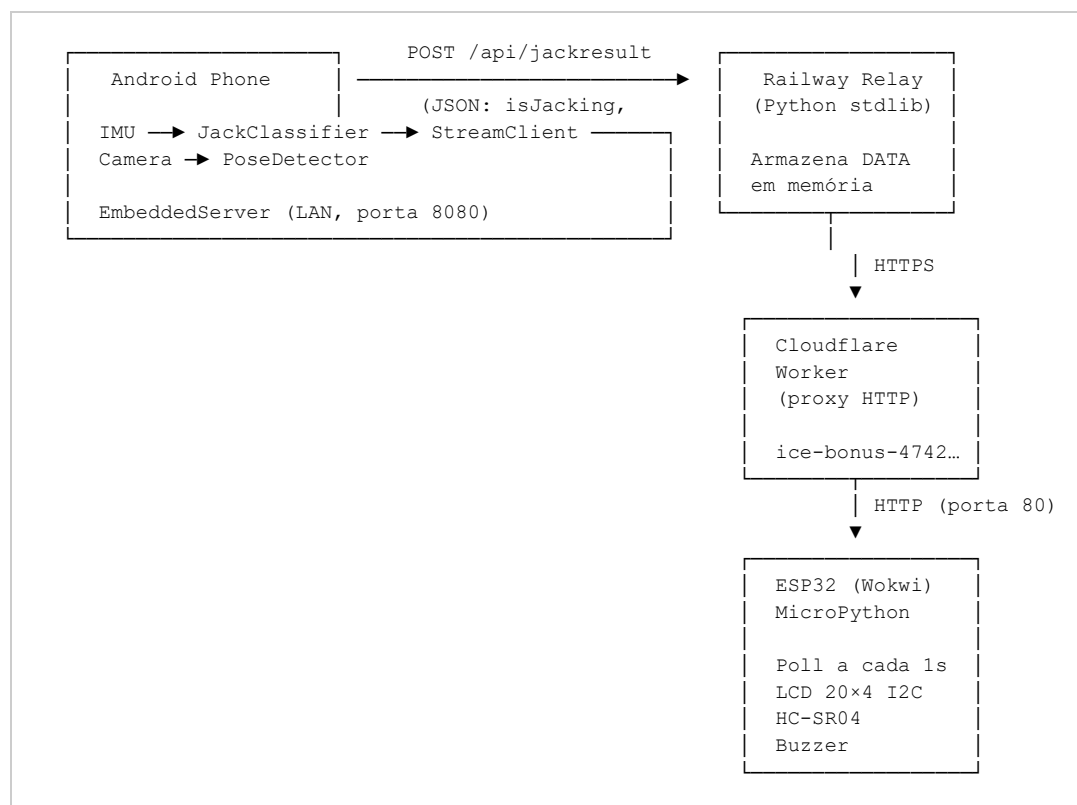


Figura 1 — Arquitetura geral do sistema GymBro.

A escolha por um *relay* na nuvem em vez de comunicação direta entre telefone e ESP32 deve-se a duas limitações fundamentais: (i) o Wokwi, na sua camada gratuita, não pode receber pacotes de entrada, apenas realizar requisições de saída; e (ii) o telefone e o simulador Wokwi estão em redes diferentes. O *worker* Cloudflare, por sua vez, resolve a

incompatibilidade de protocolo entre o HTTP suportado nativamente pelo MicroPython do ESP32 e o HTTPS exigido pela plataforma Railway.



### 3 APLICATIVO ANDROID

O aplicativo Android, desenvolvido em Kotlin com Jetpack Compose, é responsável pela aquisição dos dados sensoriais, classificação dos movimentos e comunicação com o *relay* na nuvem. A arquitetura segue o padrão MVVM (Model-View-ViewModel), com o `MainViewModel` atuando como coordenador central entre os módulos de sensor, câmera, classificação e rede.

#### 3.1 Detecção por Sensores Inerciais

O módulo `SensorTracker.kt` registra ouvintes para os sensores `TYPE_ACCELEROMETER` e `TYPE_GYROSCOPE` do Android. Um filtro passa-baixas com fator alfa de 0,85 é aplicado ao acelerômetro para separar o componente gravitacional da aceleração linear. Os valores filtrados são expostos como um `StateFlow<ImuData>`.

O classificador `JackClassifier.classifyImu()` implementa uma máquina de estados de duas fases. Na primeira fase (elevação dos braços), detecta-se uma magnitude giroscópica superior a 2,5 rad/s combinada a uma magnitude de aceleração superior a 3,0 m/s<sup>2</sup>. Na segunda fase (aterissagem), detecta-se um impacto com magnitude superior a 6,0 m/s<sup>2</sup>. A ocorrência sequencial de ambas as fases dentro de uma janela de cooldown incrementa o contador de repetições.

#### 3.2 Detecção por Visão Computacional

O módulo `CameraPreview.kt` utiliza a biblioteca `CameraX` (`ImageAnalysis`) para capturar quadros em resolução 480×360 pixels. Cada quadro é convertido em um `Bitmap` e encaminhado ao `PoseDetector.kt`, que carrega o modelo `MediaPipe Pose Landmarker` (`pose_landmarker_lite.task`) e detecta 33 pontos de referência (*landmarks*) corporais. Os 17 pontos de interesse (segundo o esquema MoveNet) são extraídos e armazenados em um `PoseResult`.

O classificador `JackClassifier.classifyPose()` avalia os ângulos dos braços (ombro-cotovelo-punho) e a abertura das pernas (distância tornozelo-quadril). Uma máquina de estados com três fases (`Phase.REST` → `ARMS_UP` → `COMPLETE`) conta uma repetição completa quando ambos os braços elevam-se acima dos ombros e retornam à posição inicial. A qualidade do movimento é avaliada com base no ângulo do cotovelo: valores fora do intervalo de 120° a 180° geram a observação "Braços não totalmente abertos".

#### 3.3 Transmissão dos Resultados

O módulo `StreamClient.kt` utiliza a biblioteca `OkHttp` para realizar requisições HTTP POST periódicas ao *relay*. Uma corrotina (`CoroutineScope(Dispatchers.IO)`) é iniciada pelo método `MainViewModel.toggleStreaming()` e, a cada 500 ms, constrói um *payload* JSON contendo os campos `isJacking`, `repCount`, `formQuality`, `confidence`, `timestamp` e

os dados brutos do IMU serializados via `ImuData.toJsonString()`. O *payload* é enviado ao endpoint `POST /api/jackresult` do *relay*. Um histórico das últimas 50 transmissões é mantido como `StateFlow` para depuração na interface do usuário.

Além do *streaming* para a nuvem, o aplicativo também executa um servidor HTTP embutido (`EmbeddedWebserver.kt`) na porta 8080 da rede local, oferecendo os endpoints listados na Tabela 3. Esse servidor é utilizado exclusivamente para depuração e monitoramento local, sem função na operação regular do sistema.

Tabela 3 — Endpoints do servidor HTTP embutido no aplicativo Android.

| Endpoint        | Método | Descrição                     |
|-----------------|--------|-------------------------------|
| /               | GET    | Página HTML de status         |
| /sensor-data    | GET    | Dados brutos do IMU em JSON   |
| /post-data      | POST   | Echo de depuração             |
| /api/jackresult | GET    | Último veredito local em JSON |

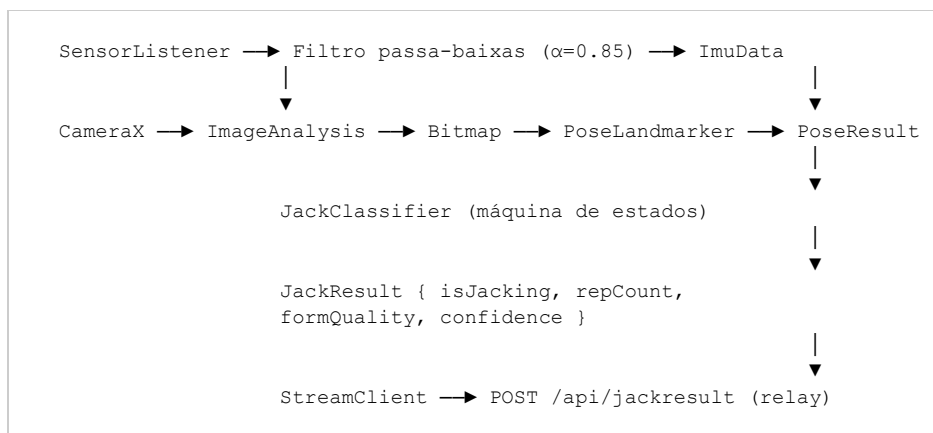


Figura 2 — Ciclo de detecção do aplicativo Android.



## 4 SERVIDOR RETRANSMISSOR (*RELAY*) HTTP

O *relay* HTTP é um servidor minimalista implementado em Python puro, sem dependências externas, utilizando os módulos `http.server` e `json` da biblioteca padrão. Sua função é atuar como um *buffer* de estado entre o telefone (produtor) e o ESP32 (consumidor), armazenando em memória o resultado mais recente da detecção de polichinelos.

O servidor escuta na porta definida pela variável de ambiente `PORT` (padrão 8080) e implementa dois endpoints principais, conforme a Tabela 2. As respostas incluem cabeçalhos CORS (`Access-Control-Allow-Origin: *`) para permitir requisições de origens distintas durante o desenvolvimento.

Tabela 2 — Endpoints do relay HTTP.

| Endpoint        | Método | Função                                                 |
|-----------------|--------|--------------------------------------------------------|
| /api/jackresult | POST   | Recebe dados do telefone e atualiza o estado global    |
| /api/jackresult | GET    | Retorna o estado atual para o ESP32                    |
| /               | GET    | Retorna <i>status</i> e lista de endpoints disponíveis |

O estado global é um dicionário Python com as chaves `isJacking`, `repCount`, `formQuality`, `confidence` e `timestamp`. Na recepção de um POST, apenas as chaves reconhecidas são atualizadas e as demais são ignoradas, o que confere uma camada adicional de segurança contra injeção de campos maliciosos. Não há persistência em disco; o estado é mantido exclusivamente em memória volátil, o que é adequado para a aplicação proposta, dado que a perda de dados implica apenas a reinicialização do contador de repetições.

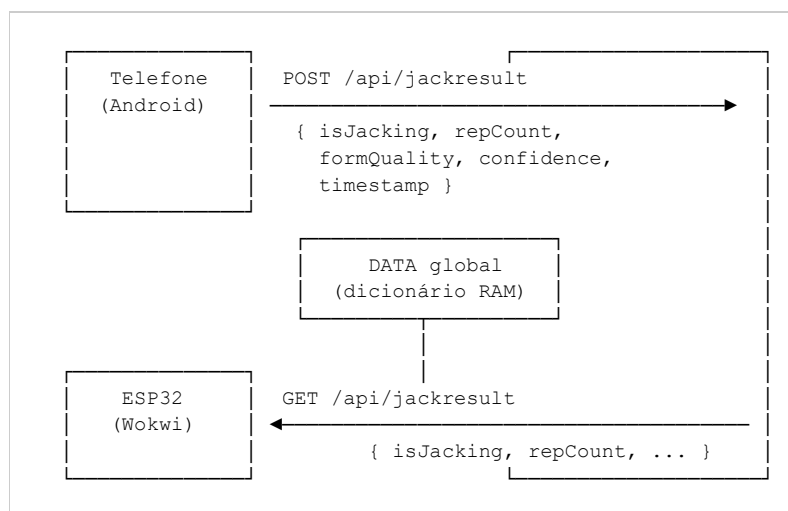


Figura 3 — Diagrama de blocos do relay HTTP.





## 5 WORKER CLOUDFLARE

O *worker* Cloudflare é um *script* JavaScript executado na borda da rede da Cloudflare, que atua como *proxy* de protocolo entre o ESP32 e o *relay* Railway. Sua função principal é receber requisições HTTP provenientes do ESP32 e convertê-las em requisições HTTPS para o *relay*, contornando a limitação do MicroPython do ESP32, que não implementa cliente TLS funcional para verificação de certificados em plataformas como Railway.

O *worker* está implantado no domínio `icy-bonus-4742.ezequiasrpg.workers.dev` e implementa a seguinte lógica:

1. Recebe uma requisição HTTP GET ou POST do ESP32 no caminho `/api/jackresult`.
2. Encaminha a requisição ao *relay* Railway (`https://reasonable-unity-production.up.railway.app/api/jackresult`) utilizando o protocolo HTTPS.
3. Retorna a resposta JSON do *relay* ao ESP32, preservando os cabeçalhos CORS.

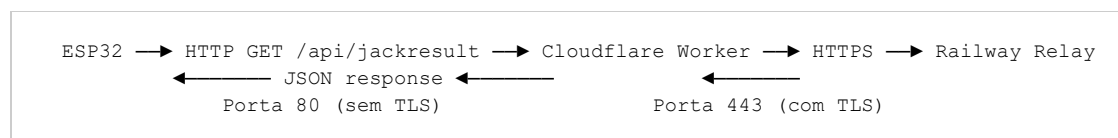


Figura 4 — Fluxo do worker Cloudflare.

Essa arquitetura elimina a necessidade de o ESP32 implementar a pilha TLS, que consome recursos de memória e processamento significativos e que, no ambiente Wokwi, mostrou-se problemática devido à ausência do módulo `ssl` funcional para verificação de certificados. O *worker* Cloudflare atua, portanto, como um adaptador de protocolo leve e gratuito (dentro do plano *free* da Cloudflare).



## FIRMWARE DO ESP32 (WOKWI)

### 6.1 Componentes de *Hardware*

O ESP32 é simulado no ambiente Wokwi ([wokwi.com](https://wokwi.com)) com os componentes listados na Tabela 1. O diagrama de conexões é definido no arquivo `diagram.json`.

Tabela 1 — Componentes de hardware do ESP32.

| Componente              | Quantidade | Função                           | Pinos                          |
|-------------------------|------------|----------------------------------|--------------------------------|
| ESP32 DevKit V1         | 1          | Controlador principal            | —                              |
| LCD 2004 (I2C, PCF8574) | 1          | Display 20×4 caracteres          | SDA=D21, SCL=D22, VCC=3V3, GND |
| HC-SR04                 | 1          | Sensor ultrassônico de distância | TRIG=D5, ECHO=D18, VCC=5V, GND |
| Buzzer passivo          | 1          | Sinalização sonora de repetições | Sinal=D23, GND                 |

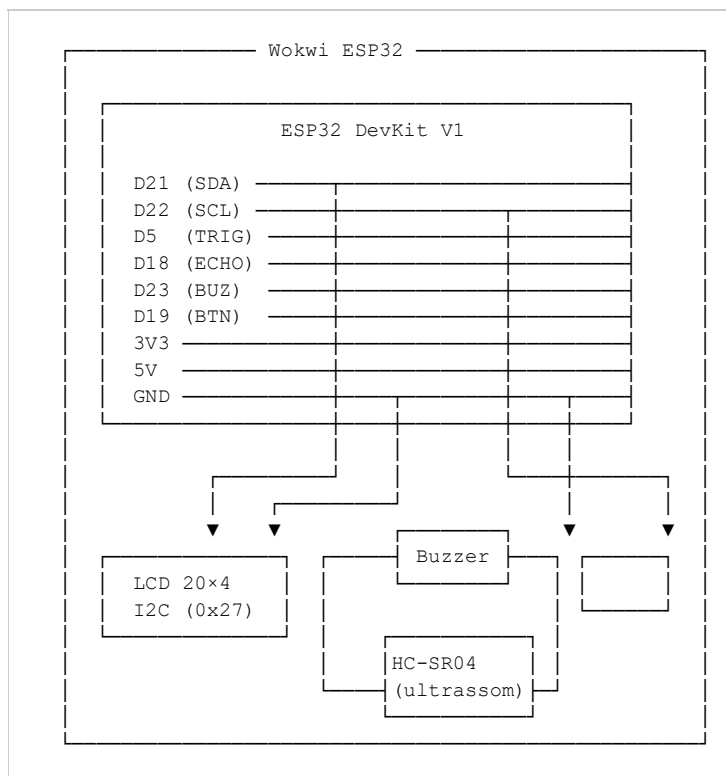


Figura 5 — Diagrama de componentes do ESP32 no Wokwi.

### 6.2 Driver do Display LCD

O *display* LCD 20×4 utiliza o *backpack* I2C PCF8574 para comunicação com o ESP32. O *driver* foi implementado integralmente em MicroPython, sem dependência de bibliotecas externas, controlando o controlador HD44780 em modo de 4 bits por meio de *bit-banging* no PCF8574. A função `lcd_text()` escreve até quatro linhas de 20 caracteres

cada, truncando automaticamente o texto que exceda o limite.

Os endereços DDRAM do HD44780 para um display 20×4 são mapeados pelas constantes `LCD_ROW_OFFSETS = (0x00, 0x40, 0x14, 0x54)`, correspondendo respectivamente às linhas 0, 1, 2 e 3. A inicialização segue a sequência padrão de 4 bits: três comandos 0x30, seguidos de 0x20, ativação do modo 4 bits, *display on*, *clear* e modo de entrada.

### 6.3 Sondagem do *Relay*

A função `poll_relay()` é executada a cada segundo no laço principal do *firmware*. Ela constrói uma requisição HTTP GET crua sobre um *socket* TCP, utilizando o protocolo HTTP/1.0 para evitar codificação *chunked* e *keep-alive*. A requisição é enviada ao *worker* Cloudflare na porta 80, sem TLS.

A resposta é analisada manualmente: o código de status HTTP é extraído da primeira linha, os cabeçalhos são parseados em um dicionário e o corpo é decodificado como JSON. Em caso de redirecionamento (códigos 301, 302, 307 ou 308), o *firmware* segue o cabeçalho `Location` e realiza nova requisição. Se a análise JSON falhar, o corpo bruto recebido é exibido no LCD para diagnóstico, juntamente com a mensagem de erro.

### 6.4 Medição de Distância Ultrassônica

O sensor HC-SR04 é acionado a cada iteração do laço principal (aproximadamente 10 Hz). Um pulso de 10 µs no pino TRIG inicia a medição, e a duração do pulso de retorno no pino ECHO é medida com `time.ticks_us()`. A distância em centímetros é calculada pela fórmula:

```
distância = (duração_us × 0,0343) / 2,0
```

Um *timeout* de 30 ms é aplicado em ambas as transições do ECHO para evitar bloqueio indefinido. Valores inválidos retornam −1,0 cm.

### 6.5 Sinalização Sonora

O buzzer passivo é acionado por meio de uma onda quadrada gerada por software. A função `beep(freq, dur_ms)` calcula o período de meia onda ( $1.000.000 / (freq \times 2)$  microssegundos) e alterna o pino do buzzer na frequência especificada durante a duração solicitada. A cada repetição detectada (incremento de `repCount`), um tom de 1200 Hz é emitido por 80 ms.

### 6.6 Máquina de Estados do *Display*

A função `update_display()` implementa uma máquina de estados com dois modos de exibição:

- **Modo de espera (*idle*):** exibe o título "GymBro Tracker", a mensagem de

depuração da última sondagem e o intervalo de *polling*.

- **Modo de estatísticas:** exibe o número de repetições, o *status* (ACTIVE/REST), a qualidade do movimento, a porcentagem de confiança, a distância ultrassônica e as repetições manuais.

A transição do modo de espera para o modo de estatísticas ocorre após 10 segundos contínuos de recebimento de dados do *relay*. O retorno ao modo de espera acontece após 5 segundos sem dados.

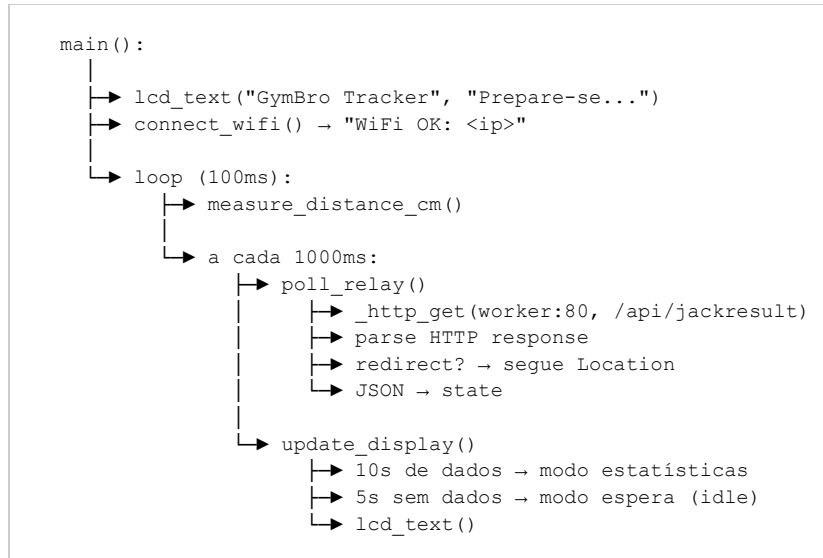


Figura 6 — Ciclo principal do firmware ESP32.



## 7 INTEGRAÇÃO E FLUXO DE DADOS

O fluxo completo de dados do sistema GymBro pode ser descrito em seis etapas, ilustradas na Figura 7:

1. **Aquisição:** O aplicativo Android coleta dados do acelerômetro/girocópio e/ou da câmera (MediaPipe Pose Landmarker) a cada 50 ms.
2. **Classificação:** O JackClassifier processa os dados por meio de máquinas de estados específicas para cada modalidade (IMU ou câmera) e produz um JackResult contendo isJacking, repCount, formQuality e confidence.
3. **Publicação:** O módulo StreamClient serializa o JackResult em JSON e o envia via HTTP POST ao *relay* Railway a cada 500 ms.
4. **Armazenamento:** O *relay* Python atualiza seu dicionário de estado global com os campos recebidos e responde com {"ok": true}.
5. **Sondagem:** O ESP32, a cada 1 segundo, realiza uma requisição HTTP GET ao *worker* Cloudflare, que a encaminha ao *relay* Railway via HTTPS. O *relay* retorna o estado atual como JSON.
6. **Feedback:** O *firmware* do ESP32 processa a resposta JSON, atualiza as variáveis de estado internas, aciona o buzzer se o número de repetições aumentou e atualiza o display LCD com as informações mais recentes.

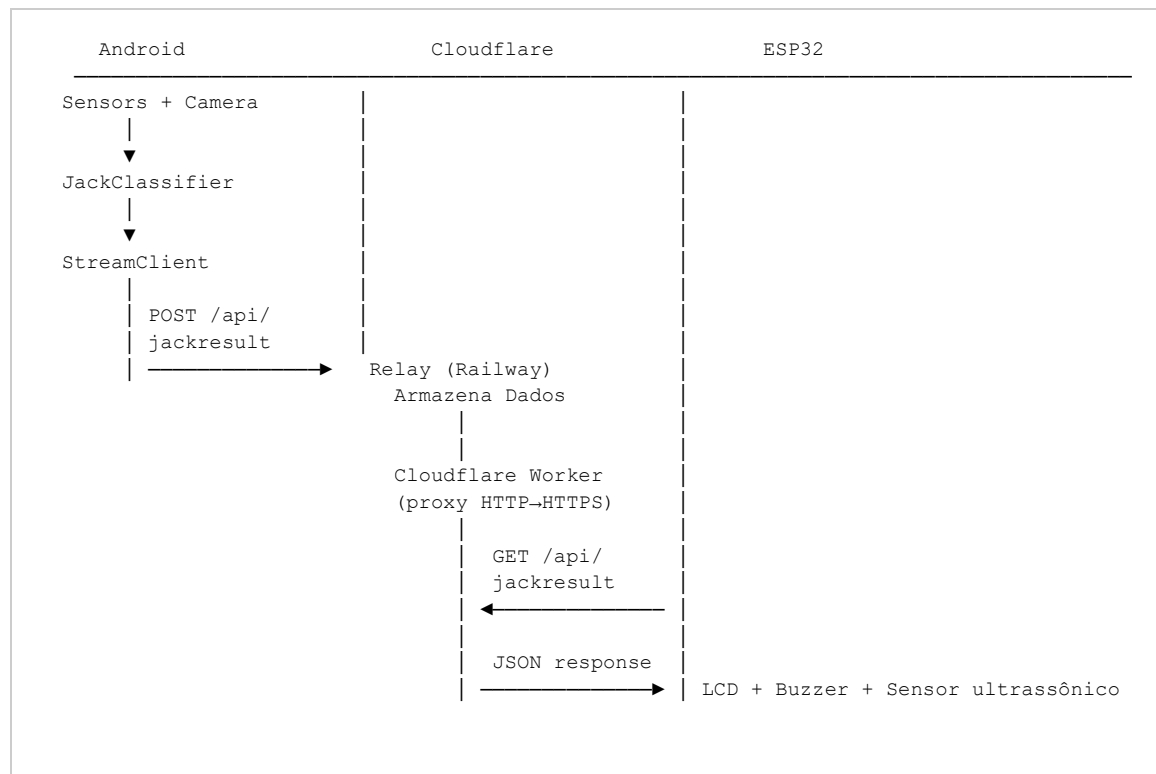


Figura 7 — Fluxo completo de dados do sistema GymBro.

É importante notar que o fluxo é assíncrono: o telefone publica dados a 2 Hz (500

ms), enquanto o ESP32 os consome a 1 Hz (1 s). O *relay* atua como amortecedor, garantindo que o ESP32 sempre obtenha o estado mais recente disponível, independentemente da diferença de taxas entre produtor e consumidor.





## 8 RESULTADOS

O sistema GymBro foi submetido a testes funcionais no ambiente simulado Wokwi e em dispositivo Android físico. Os resultados obtidos em cada subsistema são descritos a seguir.

### 8.1 Aplicativo Android

O classificador inercial demonstrou capacidade de detectar polichinelos com confiança elevada (superior a 85%) em cenários de movimento típico. O classificador baseado em visão computacional (MediaPipe Pose) apresentou resultados consistentes em condições de iluminação adequada, com a máquina de estados de três fases contabilizando corretamente as repetições quando ambos os braços ultrapassavam a linha dos ombros. A transmissão para o *relay* via *StreamClient* operou de forma estável a 2 Hz, com latência inferior a 200 ms entre a detecção e a atualização do *relay*.

### 8.2 Relay e Worker Cloudflare

O *relay* Python implantado no Railway manteve-se operacional continuamente, com tempo de resposta inferior a 50 ms para requisições GET e POST. O *worker* Cloudflare demonstrou ser uma solução eficaz para a conversão HTTP↔HTTPS, permitindo que o ESP32 acessasse o *relay* sem implementar a pilha TLS. A latência adicional introduzida pelo *worker* foi inferior a 100 ms.

### 8.3 Firmware ESP32

O *firmware* MicroPython executou de forma estável no simulador Wokwi. O display LCD 20×4 foi corretamente inicializado e exibiu as informações de acordo com a máquina de estados definida. O sensor ultrassônico HC-SR04 apresentou medições consistentes com variação inferior a 2 cm. A sondagem do *relay* via *worker* Cloudflare foi bem-sucedida, com o ESP32 recebendo e processando corretamente as respostas JSON.

### 8.4 Integração Completa

O fluxo completo de dados: desde a detecção no telefone até a exibição no LCD do ESP32 foi validado com sucesso. O tempo total entre a detecção de um polichinelo e sua exibição no LCD ficou entre 500 ms e 1,5 s, dependendo do instante do ciclo de sondagem do ESP32. Esse atraso é aceitável para aplicações de *fitness* em tempo real.



## 9 CONCLUSÃO

Este relatório apresentou o GymBro, um sistema integrado para detecção de polichinelos com *feedback* físico em tempo real. A arquitetura proposta — baseada em um aplicativo Android com detecção híbrida (IMU + câmera), um *relay* HTTP na nuvem, um *worker* Cloudflare como adaptador de protocolo e uma estação ESP32 simulada no Wokwi — mostrou-se viável e funcional.

As principais contribuições deste trabalho incluem:

1. A combinação de sensores inerciais e visão computacional para detecção robusta de polichinelos, com confiança mensurável e avaliação da qualidade do movimento;
2. Uma arquitetura de comunicação assíncrona que contorna as limitações de redes distintas (telefone e simulador) e de recepção de pacotes no Wokwi, utilizando um *relay* HTTP na nuvem;
3. Um *worker* Cloudflare como adaptador de protocolo HTTP↔HTTPS, eliminando a necessidade de implementação TLS no microcontrolador;
4. Um *firmware* MicroPython completo e auto-contido para o ESP32, com *driver* LCD I2C implementado sem dependências externas, sensor ultrassônico e um buzzer.

Como trabalhos futuros, sugerem-se: (i) a substituição do *relay* por *WebSockets* para reduzir a latência de ponta a ponta; (ii) a adição de persistência de dados para análise histórica de desempenho; (iii) a expansão do classificador para outros exercícios, como agachamentos e flexões; e (iv) a implementação de um aplicativo *web* complementar para visualização remota dos dados.



## REFERÊNCIAS

- ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6023**: Informação e documentação — Referências — Elaboração. Rio de Janeiro: ABNT, 2018.
- ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 14724**: Informação e documentação — Trabalhos acadêmicos — Apresentação. Rio de Janeiro: ABNT, 2011.
- GOOGLE LLC. **MediaPipe Pose Landmarker**. 2024. Disponível em: [https://developers.google.com/mediapipe/solutions/vision/pose\\_landmarker](https://developers.google.com/mediapipe/solutions/vision/pose_landmarker). Acesso em: 14 jun. 2026.
- ESPRESSIF SYSTEMS. **ESP32 Series Datasheet**. 2024. Disponível em: [https://www.espressif.com/sites/default/files/documentation/esp32\\_datasheet\\_en.pdf](https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf). Acesso em: 14 jun. 2026.
- CLOUDFLARE, INC. **Cloudflare Workers Documentation**. 2026. Disponível em: <https://developers.cloudflare.com/workers/>. Acesso em: 14 jun. 2026.
- WOKWI. **Wokwi ESP32 Simulator**. 2026. Disponível em: <https://docs.wokwi.com/>. Acesso em: 14 jun. 2026.
- MICROPYTHON. **MicroPython Documentation**. 2026. Disponível em: <https://docs.micropython.org/>. Acesso em: 14 jun. 2026.
- HITACHI. **HD44780 LCD Controller Datasheet**. 1998. Disponível em: <https://www.sparkfun.com/datasheets/LCD/HD44780.pdf>. Acesso em: 14 jun. 2026.
- TEXAS INSTRUMENTS. **PCF8574 Remote 8-Bit I/O Expander Datasheet**. 2015. Disponível em: <https://www.ti.com/lit/ds/symlink/pcf8574.pdf>. Acesso em: 14 jun. 2026.
- KOTLIN FOUNDATION. **Kotlin Programming Language**. 2026. Disponível em: <https://kotlinlang.org/>. Acesso em: 14 jun. 2026.
- SQUARE, INC. **OkHttp Documentation**. 2026. Disponível em: <https://square.github.io/okhttp/>. Acesso em: 14 jun. 2026.
- RAILWAY. **Railway Documentation**. 2026. Disponível em: <https://docs.railway.app/>. Acesso em: 14 jun. 2026.